

Lecture Notes: Introduction to Matplotlib (with Python & NumPy)

1. What is Matplotlib?

Matplotlib is a powerful plotting library in Python used to create:

- Static plots (images)
- Animated plots
- Interactive visualizations

It works very well with other libraries like **NumPy** and **pandas**, and is widely used for:

- Data analysis & exploration
- Scientific computing
- Creating charts and graphs for reports & presentations

You'll mainly use the submodule `matplotlib.pyplot`, commonly imported as `plt`.

2. Why Use Matplotlib? (Merits)

Some key advantages:

- **Versatile**
 - Supports line plots, bar charts, scatter plots, histograms, pie charts, etc.
- **Highly customizable**
 - Control colors, line styles, fonts, figure size, axes, titles, labels, grids, legends, and much more.
- **Works well with NumPy & pandas**

- You can directly plot arrays or Series.
 - **Widely used & well-documented**
 - Lots of tutorials, examples, and community support.
-

3. Basic Setup

Before plotting, we usually import:

```
import matplotlib.pyplot as plt
import numpy as np

%matplotlib inline # (Jupyter only) show plots inside the notebook
```

- `matplotlib.pyplot` is imported as `plt` by convention.
 - `numpy` (imported as `np`) is used to generate and manipulate numerical data.
 - `%matplotlib inline` is a **Jupyter magic command** that makes plots appear directly in the notebook.
-

4. Creating a Simple Plot (Sine Wave Example)

4.1 Prepare the Data

Use NumPy to create data points:

```
x = np.linspace(0, 10, 100) # 100 points between 0 and 10
y = np.sin(x)               # sine of each x value
```

- `np.linspace(start, stop, num)` generates evenly spaced values.
- Here, `y` is the sine of `x`, so we will get a nice wave.

4.2 Create the Figure and Axes

```
fig, ax = plt.subplots()
```

- `fig` is the **Figure** (the whole canvas).
- `ax` is the **Axes** (the actual plot area where the data is drawn).

4.3 Plot the Data

```
ax.plot(x, y, label='Sine')
```

- `ax.plot(x, y)` draws a line plot.
- `label='Sine'` will be used if we add a legend.

4.4 Customize the Plot

```
ax.set_title("Simple Sine Wave")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")
```

- `set_title()` → adds a title at the top.
- `set_xlabel()` and `set_ylabel()` → label the axes.

You can also add:

```
ax.grid(True)    # add a grid
ax.legend()      # show labels for plotted lines
```

4.5 Show the Plot

```
plt.show()
```

This renders the final figure.

What Happens When You Uncomment?

1. Add the cosine line

```
ax.plot(x, y2, color='red', linestyle='--', label='Cosine')
```

- Adds a second line (cosine) in red with a dashed style.
- Now you'll see both sine and cosine on the same axes.

2. Add the grid

```
ax.grid(True)
```

- Draws a grid in the background.
- Helps in reading exact values and understanding the shape.

3. Add the legend

```
ax.legend()
```

- Shows a small box (usually top-right) labeling each line using the `label=` arguments you provided.

This is a great way to learn: **change one thing at a time and re-run the cell to see the effect.**

6. Saving the Plot

You can save your plot to an image file.

6.1 Basic Save Command

If you have a `fig` object:

```
fig.savefig('my_plot.png')
```

- Saves the current figure as `my_plot.png` in your working directory.
- You can also save as `.jpg`, `.pdf`, `.svg`, etc.

Examples:

```
fig.savefig('sine_wave.jpg')  
fig.savefig('sine_and_cosine.pdf')
```

In many environments (like Jupyter), there's also a **save button** in the toolbar, but `savefig()` is more flexible and scriptable.
